REACT REDUX REFRESH

Day 1 to Day 6 Interview Guide

A printable study guide built from your full progress so far, from Day 1 through Day 6, March 12, 2026. It covers React fundamentals, TypeScript, routing, local state, performance, Tailwind, feature architecture, Redux Toolkit, async thunks, normalized state, selectors, Redux DevTools, and testing strategy.

Coverage: Day 1, Day 2, Day 3, Day 4, Day 5, and Day 6 work on Redux testing, mocked API boundaries, and UI integration confidence. Update this file after each new learning day and regenerate the PDF.

1. What You Should Be Able To Explain

Core React

- How props flow down and events flow up.
- How state updates trigger re-renders.
- Why controlled inputs are predictable.
- How reconciliation uses element type and keys.
- When memoization helps and when it is noise.

State Architecture

- What belongs in local component state.
- What belongs in shared global state.
- Why selectors are preferable to ad hoc derivation everywhere.
- Why normalized state scales better than large nested arrays.
- Why async state needs loading and error modeling.
- How Redux Toolkit reduces accidental complexity.
- How to test reducers, thunks, and Redux-connected UI separately.

2. Roadmap Through Today

Day	Main focus	What changed in the project
Day 1	React foundations	Built the task app with routing, local state, controlled forms, and basic localStorage persistence.
Day 2	Performance and reuse	Added reusable components, Tailwind styling, and practical memoization with `React.memo`, `useMemo`, and `useCallback`.
Day 3	Redux Toolkit basics	Moved shared task state into Redux Toolkit with slices, typed hooks, and selector-driven reads.
Day 4	Async state	Introduced `createAsyncThunk`, loading and error modeling, and a storage service boundary for async flows.
Day 5	Advanced Redux patterns	Normalized task state with entity adapter, split selectors and thunks into dedicated modules, improved async boundaries, and configured Redux DevTools tracing.
Day 6	Testing Redux and UI	Added Vitest and React Testing Library, wrote reducer and thunk tests, mocked the storage API boundary, tested form behavior, and verified Redux + UI flows together.

3. Day 1 Foundations

Component Model

A React component is a function that maps state and props to UI. It is not a template file plus code-behind. Every render is a fresh execution of the component function. Variables declared inside the component are recalculated on each render unless persisted through hooks or lifted out of the render path.

Props, Events, and Data Flow

```
Parent state
  -> passes props to child
Child reads props
  -> user interacts
Child calls callback prop
  -> parent updates state
Parent re-renders
  -> updated props flow down again
```

This is the default React model: unidirectional data flow. The parent owns the state, children receive values and functions. Senior-level reasoning here is about keeping ownership explicit and not scattering mutable state across unrelated components.

Controlled Inputs

A controlled input binds `value` to React state and updates that state in `onChange`. The source of truth becomes the component state instead of the DOM node. This makes validation, formatting, submit handling, and reset logic deterministic.

> **Interview question:** Why use controlled inputs instead of uncontrolled inputs?
>
> **Answer:** Controlled inputs make state transitions explicit, which improves validation, serialization, reset behavior, and coordination with other UI state. Uncontrolled inputs can be simpler for basic forms, but controlled inputs are the standard choice when the form behavior matters.

Local Storage Persistence

Local storage is synchronous and browser-local. It works for demos, simple offline persistence, and learning projects, but it is not a substitute for a real server. It has no conflict resolution, auth, schema evolution, or concurrent-user guarantees.

React Router Basics

Router configuration maps URLs to UI. At interview level, explain that route state should be used for navigation concerns, not as a generic replacement for application state. Route params identify resources; app state represents current client data.

4. React Internals You Should Know

Concept	What it means	Why interviewers care
Render phase	React computes the next UI tree by calling components.	Shows you understand render is calculation, not DOM mutation.

Concept	What it means	Why interviewers care
Commit phase	React applies the minimal DOM updates after diffing.	Helps explain why pure rendering matters.
Reconciliation	React compares previous and next trees.	Explains keys, preservation of state, and remount behavior.
Batching	Multiple state updates can be grouped into one render.	Shows you know React is scheduling work, not immediately mutating UI.
Closures	Handlers capture values from the render they were created in.	Important for stale state bugs and hook dependencies.

> **Interview question:** Why does React need keys in lists?
>
> **Answer:** Keys let React track identity across renders. Without stable keys, React may reuse the wrong DOM nodes or component instances, causing incorrect state preservation and unnecessary remounting.

> **Interview question:** What causes a React component to re-render?
>
> **Answer:** A re-render happens when its local state changes, its parent re-renders and passes new props, the component consumes changed context, or a subscribed store selector returns a changed value.

5. How Params and State Move Through a React App

Props

Use props when state ownership is local to a parent and children only need values or callbacks. This is the cleanest path because the dependency graph stays obvious.

```
type TaskFormProps = {
  onAdd: (title: string) => void;
  disabled?: boolean;
};
```

Local State

Use `useState` for UI concerns that are local and temporary: input text, modal open/closed state, touched flags, active tab, or draft values that do not need broad sharing.

Route Params

Use route params to identify the resource the screen should display: `/tasks/:taskId`. Do not put large mutable UI state in the URL unless shareability and navigation semantics justify it.

Redux Store

Use Redux when multiple parts of the app depend on the same state, when update logic should be centralized, or when async request status needs a consistent model. Components should read store state through selectors, not by repeatedly digging through the store shape inline.

```
UI event
  -> dispatch(thunk or action)
Store processes reducer / extraReducer
  -> new state produced
Selector reads derived result
  -> subscribed component re-renders
```

How To Choose

Mechanism	Use when	Avoid when
Props	State has a clear nearby owner.	Too many intermediate components are forwarding data.
Local state	The data is view-local and short-lived.	Other screens or features need it.
Route params	The URL should identify current resource or filter.	The data is ephemeral UI detail.
Redux	Multiple consumers, shared async status, centralized rules.	The state is purely local and simple.

6. Day 2 Performance and Reuse

`React.memo`

`React.memo` skips re-rendering a child when its props are shallowly equal. It is useful when the child is expensive or receives stable props. It is not a default wrapper for every component.

`useMemo`

Use `useMemo` for expensive derived values or to maintain stable references when reference identity matters. Do not use it to hide cheap calculations or as cargo-cult optimization.

`useCallback`

`useCallback` memoizes function identity. It matters when a child is memoized or when a dependency array would otherwise change on every render. It does not make the function faster to execute.

> **Interview question:** When is `useCallback` useless?
>
> **Answer:** It is usually useless when the callback is cheap, the child is not memoized, and function identity is not part of any dependency-sensitive path. In those cases, it adds code without reducing work.

Feature-Based Architecture

A feature folder groups components, state, utilities, and types by business domain rather than by technical layer. This improves local reasoning, reduces cross-folder hunting, and makes ownership clearer.

Tailwind CSS

Tailwind is utility-first styling. Senior-level usage means being able to discuss tradeoffs: fast iteration and predictable colocation versus large class strings and the need for disciplined abstractions when repeated patterns emerge.

7. TypeScript in a React Codebase

- Type props at the component boundary.
- Type event handlers where DOM event detail matters.
- Type Redux hooks so selectors and dispatch stay safe.
- Use unions for finite UI states such as filters and request status.
- Prefer explicit domain types over `any`.

> **Interview question:** Why is `any` dangerous in frontend code?
>
> **Answer:** It disables the compiler exactly at the boundary where bugs are likely: props, API data, reducer payloads, and event handling. It lowers trust in refactors and makes runtime issues easier to ship.

8. Day 3 Redux Toolkit

Store, Slice, Reducer, Selector

Redux Toolkit provides structured primitives for predictable state management. A store holds application state. A slice defines a domain of state plus reducers. Selectors are read APIs over store state. This separation matters because it prevents UI components from becoming the place where every rule lives.

Immer and Immutable Updates

Redux Toolkit uses Immer under the hood, allowing reducer code that looks mutative while producing immutable results. The practical benefit is less boilerplate and lower risk of accidental mutation bugs.

```
state.items.unshift(newTask)

// looks mutable, but Immer tracks the draft and
// returns a safe immutable next state
```

Why Selectors Matter

Selectors centralize read logic. If every component calculates visible tasks, stats, or derived flags separately, business logic becomes duplicated and inconsistent. Selectors also support memoization.

> **Interview question:** What problem does Redux Toolkit solve over plain Redux?
>
> **Answer:** It removes repetitive setup, encourages sane defaults, bakes in Immer, simplifies async with thunks, and improves maintainability. It solves accidental ceremony more than conceptual complexity.

9. Day 4 Async Thunks and Request State

Why Async Changes the Design

Once data is asynchronous, the UI can no longer assume a value is simply present. You must model at least four states: idle, loading, succeeded, and failed. Without this, your app will show stale screens, broken controls, or silent failures.

`createAsyncThunk` Lifecycle

```
dispatch(fetchTasks())
  -> pending
  -> async work runs
  -> fulfilled with payload
     or rejected with error
```

A thunk is not just "an async function in Redux." It is a structured flow that gives reducers lifecycle events, lets UI track request state, and keeps side effects outside presentational components.

Senior-Level Concerns in Async UI

- Prevent overlapping writes when the UI cannot safely merge them.
- Show clear loading and error states.
- Keep reducer logic deterministic.
- Model retries intentionally, not accidentally.
- Know when local optimistic updates are safe and when they are not.

> **Interview question:** Why store request status in Redux state?
>
> **Answer:** Because loading, success, and failure are part of the user-visible state of the feature. Multiple components may need to react consistently to those states, so centralizing them avoids fragmented UI behavior.

> **Interview question:** What is the difference between a reducer and a thunk?
>
> **Answer:** A reducer is a pure state transition function. A thunk orchestrates side effects and dispatches actions before and after async work. Reducers must stay deterministic; thunks are where I/O belongs.

10. Day 5 Advanced Redux Patterns

Normalized State with Entity Adapter

Storing entities as an ID map plus ordered IDs is a production pattern because reads and writes stay predictable as the collection grows. Instead of replacing an entire array after each mutation, reducers can add, update, or remove a single entity with focused operations.

```
tasks: {
  ids: ["t3", "t2", "t1"],
  entities: {
    t1: { id: "t1", title: "Ship Redux refactor", completed: false },
    t2: { id: "t2", title: "Write selectors", completed: true },
    t3: { id: "t3", title: "Inspect DevTools", completed: false }
```

```
  }
}
```

Selectors as the Read API

Components should not know storage shape details. A selector layer becomes the read API for the feature: visible task IDs, counts, loading flags, and derived error state. Memoization matters when selectors are reused and when their outputs would otherwise allocate new arrays or objects every render.

> **Interview question:** Why memoize selectors instead of computing filtered lists inside components?
>
> **Answer:** Because selector memoization centralizes derivation, avoids repeated allocation work for identical inputs, and keeps components focused on rendering instead of reimplementing domain rules.

Cleaner Separation of Concerns

A maintainable Redux feature usually separates write orchestration, state transitions, and read logic:

- `tasksThunks.ts` handles async orchestration and error translation.
- `tasksSlice.ts` handles state transitions.
- `tasksSelectors.ts` handles derived read logic.
- `storage.ts` acts as the persistence boundary.

Better Async Patterns

Production thinking means modeling fetch and mutation status separately, preventing overlapping writes when the UI cannot safely merge them, and preserving useful failure messages with `rejectWithValue`. This makes the UI and DevTools timeline easier to reason about.

Redux DevTools as an Architecture Tool

DevTools is not just for debugging bugs after the fact. It helps verify that actions are small, reducers are deterministic, thunk lifecycles are visible, and state shape stays understandable. If your DevTools history looks noisy or confusing, the architecture often needs work.

What Today's Refactor Proves

- You can treat selectors as the stable read API for a feature.
- You can update one entity without replacing the entire collection.
- You can separate fetch status from mutation status instead of collapsing everything into one loading flag.
- You can keep side effects in thunks and state transitions in reducers.
- You can use DevTools to inspect architecture quality, not just bugs.

> **Interview question:** What is the benefit of entity adapter if the UI still shows a simple list?
>
> **Answer:** The UI may be simple today, but normalized state reduces update cost, improves selector composition, and avoids rewriting reducers later when detail views, lookups, or cross-feature references appear.

> **Interview question:** Why separate thunk files from slice files?
>
> **Answer:** Because async orchestration and pure state transitions are different responsibilities. Separating them reduces file sprawl, makes testing easier, and keeps reducers focused on deterministic updates.

11. Day 6 Testing Redux and UI

What Was Added in the Task App

Day 6 turned the architecture work into a testing surface. The project now has four complementary test layers:

- `tasksSlice.test.ts` verifies pure reducer transitions such as filter changes, fulfilled fetches, and rejected mutations.
- `tasksThunks.test.ts` verifies async thunk orchestration, including success paths, condition guards, and rejected payload handling.
- `TaskForm.test.tsx` verifies component behavior such as validation, trimmed input submission, and field reset behavior.
- `TasksPage.integration.test.tsx` verifies the full Redux-connected screen: initial loading, fetched tasks, adding a task, toggling a task, and last-mutation UI updates.

Why the Storage Layer Is Mocked

The task feature uses `storage.ts` as a persistence boundary. In tests, that boundary is mocked with `vi.mock` so the test can control responses without depending on real browser persistence or timer delays. That keeps tests focused on Redux and UI behavior rather than localStorage mechanics.

```
UI event
  -> dispatch(addTask or toggleTask thunk)
Thunk calls mocked storage boundary
  -> mocked success or failure response
Reducer handles pending / fulfilled / rejected
Selector output changes
  -> component re-renders with updated UI
```

How To Explain the Testing Strategy

The testing strategy is layered. Pure reducer tests prove deterministic state transitions. Thunk tests prove async orchestration and error translation. Component tests prove isolated UI behavior. Integration tests prove that Redux state, selectors, thunks, and rendering work together under user interaction.

> **Interview question:** Why unit test reducers separately if integration tests already exist?
>
> **Answer:** Reducers are pure and cheap to test, so they give fast feedback on state transition rules. Integration tests are broader and valuable, but reducer tests make it easier to pinpoint whether a failure is in state logic or in the UI wiring.

Interview question: What should an async thunk test prove?

Answer: It should prove the orchestration contract: whether the thunk calls the dependency with the right inputs, whether condition guards block invalid work, and whether fulfilled or rejected results produce the expected Redux state and payloads.

Interview question: Why mock the API or storage boundary instead of using the real implementation in every test?

Answer: Because most tests should validate application behavior, not re-test browser storage mechanics or network timing. Mocking the boundary makes tests deterministic, faster, and more focused on the code under review.

Interview question: What is the difference between a component test and a Redux + UI integration test?

Answer: A component test isolates a small UI unit and usually passes props directly. A Redux + UI integration test renders the connected screen with a real store, dispatch flow, selectors, and user events so it can validate that the feature works as a system.

Interview question: In this task app, what does the Day 6 integration test actually prove?

Answer: It proves that the page dispatches the initial fetch, renders loading and fetched task data, submits a new task through the thunk path, toggles an existing task, and reflects the last mutation in the UI. That is stronger evidence than checking one reducer or one component in isolation.

Interview question: How do you keep frontend tests from becoming brittle?

Answer: Test behavior instead of implementation details. Prefer user-visible queries, keep mocks at clear boundaries, avoid asserting incidental markup, and reserve deep internal assertions for pure units like reducers or selector logic.

12. Senior Frontend Interview Questions and Answers

Q: How do you decide whether state should live in a component, context, or Redux?

A: Start with ownership and scope. If one component owns it, keep it local. If a tree branch needs it and prop drilling becomes structural noise, context may fit. If multiple parts of the app need the data plus coordinated updates, async lifecycle, selectors, or debugging support, Redux is justified.

Q: What would make you reject a memoization optimization in code review?

A: If it is guarding cheap work, added without measurement, obscures code, or relies on unstable dependencies that invalidate it every render. Memoization should remove meaningful work, not add cognitive overhead for theoretical wins.

Q: How would you explain stale closures to a junior developer?

A: A callback remembers variables from the render where it was created. If you later expect it to see newer values without updating its dependencies or using the right pattern, it can act on old data.

Q: Why are selectors better than reading nested store paths directly in every component?

A: They centralize read logic, improve reuse, isolate store shape changes, and support memoization. Components should ask for domain data, not know too much about storage layout.

Q: How would you describe Immer in one sentence?

A: Immer lets you write reducer logic in a mutating style while safely producing immutable next state behind the scenes.

Q: What are the risks of using index as a React list key?

A: If list order changes, component identity shifts incorrectly, which can preserve the wrong local state or cause unnecessary DOM churn. Index keys are only acceptable for static, never-reordered lists.

Q: How would you discuss localStorage in a system design interview?

A: As a browser persistence primitive suitable for lightweight local caching or user convenience, but not as a source of truth for multi-user, secure, or highly consistent systems.

Q: What should a good loading state do?

A: Communicate that work is in progress, prevent broken interactions, preserve context where possible, and avoid visual jumps that confuse the user.

Q: How do you evaluate whether Redux is overkill?

A: If the shared state is tiny, short-lived, or consumed by a narrow subtree, Redux can add indirection. If the feature requires centralized updates, async modeling, selector reuse, or debugging clarity, it is often justified.

Q: What is the difference between derived state and duplicated state?

A: Derived state is computed from a source of truth when needed. Duplicated state stores the same information in multiple places, which creates synchronization bugs.

13. How To Explain Your Project in an Interview

A strong answer is structured. Do not just list libraries. Explain the problem, architectural choices, and tradeoffs.

```
I built a task management learning project in React and TypeScript over six structured days.
I started with local state and controlled forms to reinforce data flow.
Then I added Tailwind and reusable components while practicing memoization.
After that I moved shared task state into Redux Toolkit with typed hooks and selectors.
For Day 4, I introduced createAsyncThunk and modeled request status, loading, and error handling using
an async local storage layer.
For Day 5, I normalized the task state with entity adapter, split selectors and thunks into dedicated
modules, and configured Redux DevTools tracing so the state transitions are easier to inspect.
For Day 6, I added Vitest and React Testing Library, then wrote reducer tests, thunk tests with mocked
storage calls, component tests, and a Redux + UI integration test for the task page.
The main goal was not feature count; it was learning how state architecture and test strategy evolve
together as complexity grows.
```

14. Red Flags Interviewers Notice

- Using `any` at critical boundaries.
- Memoizing everything by default.
- Confusing route state with application state.
- Putting business logic in random components.
- No loading or error states for async operations.
- Using unstable keys in dynamic lists.
- Duplicating derived state in multiple places.
- Replacing whole entity collections for small mutations.
- Choosing Redux before proving shared-state need.
- Not being able to explain why a hook dependency exists.
- Writing brittle tests that assert implementation details instead of behavior.
- Using integration tests only and skipping focused unit tests for pure logic.
- Talking only about syntax instead of tradeoffs.

15. Final Revision Checklist

- Can you explain props, state, and controlled inputs clearly?
- Can you describe render, commit, reconciliation, and keys?
- Can you justify when to use `useMemo` or `useCallback`?
- Can you explain why feature-based architecture scales better?
- Can you describe store, slice, reducer, selector, and Immer?
- Can you explain `createAsyncThunk` pending, fulfilled, and rejected?
- Can you explain normalized state, IDs/entities, and entity adapter tradeoffs?
- Can you explain why selectors form the read API for Redux state?
- Can you explain how Redux DevTools helps validate architecture decisions?
- Can you explain what reducers, thunks, component tests, and integration tests each prove?
- Can you explain why mocking the API boundary improves determinism?
- Can you explain how parameters and state move through your app?
- Can you discuss tradeoffs like a senior engineer rather than just list APIs?

Suggested next update after Day 6: selector-focused tests, RTK Query tradeoffs, real HTTP integration boundaries, and end-to-end browser coverage.